

Git Workflow Guide — Kelarin Project

Dokumen ini menjelaskan standar workflow Git yang digunakan dalam pengembangan aplikasi Kelarin agar kerja tim lebih terstruktur, aman, dan kolaboratif.

1. Branch Naming Convention

Setiap perubahan harus dilakukan di branch terpisah, bukan langsung ke `main`.

Format:

```
tipe/deskripsi-singkat
```

Contoh:

- `feature/add-task`
- `fix/login-error`
- `docs/update-readme`
- `chore/update-dependencies`
- `refactor/split-crud-service`

Jenis Branch:

- **feature/** → untuk menambah fitur baru
- **fix/** → untuk memperbaiki bug
- **docs/** → untuk dokumentasi
- **chore/** → konfigurasi atau maintenance
- **refactor/** → perbaikan kode tanpa mengubah fungsi

2. Commit Message Convention

Menggunakan format **Conventional Commits**:

```
tipe: deskripsi singkat
```

Contoh:

- `feat: add task feature`
- `fix: resolve login bug`
- `docs: update API documentation`
- `chore: update docker configuration`

Jenis Commit:

- **feat** → fitur baru
 - **fix** → bug fix
 - **docs** → dokumentasi
 - **refactor** → perbaikan struktur kode
 - **chore** → maintenance
 - **test** → testing
-

3. Pull Request (PR) Process

Setiap perubahan harus melalui Pull Request sebelum masuk ke **main**.

Langkah:

1. Buat branch dari **main**
2. Lakukan perubahan
3. Commit dan push ke GitHub
4. Buat Pull Request
5. Tambahkan deskripsi perubahan
6. Assign reviewer
7. Tunggu review dan approval
8. Merge ke **main** setelah disetujui

Aturan:

- Tidak boleh push langsung ke **main**
 - Minimal 1 reviewer
 - Gunakan **Squash and Merge**
-

4. Code Review Guidelines

Code review bertujuan untuk memastikan kualitas kode sebelum digabungkan.

Hal yang diperiksa:

- Fungsionalitas berjalan dengan baik
- Tidak ada error atau bug
- Kode mudah dibaca
- Mengikuti standar project
- Tidak ada data sensitif yang hardcoded

Jenis komentar:

- 🎉 Apresiasi (apa yang sudah baik)
 - 💡 Saran perbaikan
 - ? Pertanyaan (jika ada yang kurang jelas)
-

Keputusan review:

- **Approve** → jika sudah siap di-merge
 - **Comment** → jika masih diskusi
 - **Request changes** → jika perlu perbaikan
-

5. CODEOWNERS

Project ini menggunakan file `.github/CODEOWNERS` untuk menentukan reviewer otomatis.

Pembagian:

- Backend → Lead Backend
- Frontend → Lead Frontend
- DevOps → Lead DevOps
- Dokumentasi → Lead QA & Docs

Dengan adanya CODEOWNERS:

- Reviewer otomatis ditambahkan saat PR dibuat
 - Proses review menjadi lebih cepat dan terarah
-

6. Alur Workflow Tim (PENTING)

Berikut alur kerja yang digunakan dalam pengembangan project:

main → buat branch → coding → PR → review → merge

Penjelasan:

1. Mulai dari branch main

Pastikan branch main sudah dalam kondisi terbaru dengan melakukan git pull.

Cara:

i. `git checkout main`

ii. `git pull origin main`

2. Buat branch baru

Setiap fitur atau tugas harus dikerjakan di branch terpisah.

Contoh:

- `git checkout -b feature/add-task` (jika belum ada branch dan ingin membuat branch baru)
 - `git checkout feature/add-task` (jika sudah punya branch dan ingin melakukan perubahan)
-

3. Lakukan coding

Kerjakan perubahan atau fitur di branch tersebut tanpa mengganggu branch main.

4. Commit & Push

Setelah selesai, simpan perubahan dengan commit lalu push ke repository:

Contoh

```
git add .
git commit -m "feat: add task feature"
git push origin feature/add-task
```

5. Buat Pull Request (PR)

Ajukan perubahan dengan membuat Pull Request ke branch main melalui GitHub.

6. Code Review

Anggota tim lain akan melakukan review terhadap kode yang diajukan.

Berikut pasangan review:

PR dari	Reviewer
Lead Backend	Lead Frontend
Lead Frontend	Lead Backend
Lead DevOps	Lead QA & Docs
Lead QA & Docs	Lead DevOps
Lead CI/CD	Lead Backend atau Frontend

Jika ada saran atau perbaikan, lakukan revisi terlebih dahulu.

7. Merge ke main

Jika sudah disetujui (approve), perubahan dapat di-merge ke branch main.

Catatan Penting

- Tidak diperbolehkan push langsung ke branch main
- Setiap perubahan wajib melalui Pull Request
- Minimal satu approval sebelum merge

- Gunakan penamaan branch dan commit yang jelas
-

Kesimpulan

Dengan mengikuti workflow ini, diharapkan:

- Kolaborasi tim menjadi lebih rapi
- Risiko error berkurang
- Kualitas kode lebih terjaga

Setiap anggota tim wajib mengikuti standar ini selama pengembangan project berlangsung.